

Презентація: Модуль 9, Урок 3 - Швидкість API, HTTP-кешування та підготовка до highload

Красна нить (розкривається в кінці):

«*Production-ready фінансовий API = швидкі запити до БД + розумне кешування (серверне й HTTP) + контроль об'єму даних + rate limiting + jobs. Усе інше — лише реалізаційні деталі.*»

БЛОК 1: Вхід у фінальний урок програми

Слайд 1 - Титульний

На екрані:

- Модуль 9. Основи продуктивності та оптимізації
- Урок 3. Швидкість API, HTTP-кешування та підготовка до highload
- Підзаголовок: «API як продукт: швидкість, стабільність, масштаб»

Нотатки доповідача:

Привітайте групу, окремо підкресліть, що це **завершальний урок програми**. Нагадайте:

- 9.1 — мислення продуктивністю, пошук вузьких місць;
 - 9.2 — кеш + БД (eager loading, індекси, пагінація).
- Сьогодні — фінальний шар: **швидкість API з погляду HTTP, кешу та highload**.
-

Слайд 2 - Зв'язок із попередніми модулями

На екрані:

- Заголовок: «Що ми вже маємо перед оптимізацією API»
- Тези:
 - Модулі 5–7: архітектура, DTO/Resources, REST-контракт, тести;
 - Модуль 8: логи, Sentry, лог-агрегація;
 - Модуль 9: кеш і БД (9.2) + HTTP-кеш, ліміти й highload (9.3).

Нотатки доповідача:

Поясніть, що нинішня тема опирається на все попереднє: неможливо прискорити API, якщо немає нормальної архітектури, моніторингу й роботи з БД.

Слайд 3 - Мета уроку

На екрані:

- Заголовок: «Мета уроку»
- Пункти:
 - розібрати, з чого складається час відповіді API;

- показати, як використовувати HTTP-кешування (Cache-Control, ETag);
- наголосити на контролі об'єму даних (DTO/Resources, пагінація);
- показати роль rate limiting і jobs за високого навантаження (highload);
- зібрати фінальний образ production-ready API.

Нотатки доповідача:

Поясніть, що це урок «зведення всього разом»: не тільки про мікропатерни, а про загальну картину.

БЛОК 2: Швидкість API як властивість продукту

Слайд 4 - API як вузьке місце системи

На екрані:

- Заголовок: «Повільне API = повільний продукт»
- Ідеї:
 - від API залежить веб, мобільний, інтеграції;
 - якщо бекенд дає 3 с — UX завжди 3 с;
 - у фінансах це ще й довіра: таймаут під час оплати $\approx$ втрата впевненості.

Нотатки доповідача:

Наведіть приклади: повільний список платежів, «крутиться спінер» під час підтвердження.

Слайд 5 - Що означає «швидке API»

На екрані:

- Заголовок: «Не один швидкий запит, а стабільний сервіс»
- Пункти:
 - стабільні p95/p99 (без «пиків»);
 - немає деградації зі зростанням обсягу даних (пагінація, індекси);
 - коректна робота під навантаженням (кеш, ліміти, jobs).

Нотатки доповідача:

Зв'яжіть із Модулем 8: перевіряємо це не «на око», а за метриками/логами.

Слайд 6 - З чого складається час відповіді

На екрані:

- Заголовок: «Шлях запиту в бекенді»
- Етапи:
 - мережа + прийом HTTP-запиту;
 - middleware (auth, логи, correlation_id);
 - контролер/сервіси (бізнес-логіка);
 - БД/кеш (основний «часовий» вклад);
 - серіалізація JSON (DTO/Resources);

- відправка відповіді.

Нотатки доповідача:

Поясніть, що 9.2 фокусувався на БД/кеші, а сьогодні додаємо HTTP-шар і highload-аспекти.

БЛОК 3: HTTP як рівень оптимізації — кешування

Слайд 7 - HTTP як не лише «транспорт для JSON»

На екрані:

- Заголовок: «Що дає нам HTTP»
- Можливості:
 - Cache-Control, ETag, Last-Modified;
 - статус 304 Not Modified;
 - прокси/браузерний кеш.

Нотатки доповідача:

Поясніть, що ігнорувати HTTP-кеш — втрачати безкоштовний приріст швидкості на повторних запитах.

Слайд 8 - Що таке HTTP-кешування

На екрані:

- Заголовок: «Кеш у клієнта/проксі, не в нашому коді»
- Ідеї:
 - ресурс кешується в браузері/CDN;
 - за повторного запиту — умовний запит (**If-None-Match/If-Modified-Since**);
 - якщо не змінився — 304 без тіла.

Нотатки доповідача:

Підкресліть, що це зменшує навантаження й трафік, а також покращує відгук для користувача.

Слайд 9 - Cache-Control та ETag (оглядово)

На екрані:

- Заголовок: «Основні заголовки HTTP-кешу»
- **Cache-Control**:
 - **public, max-age=60** — можна кешувати де завгодно 60 с;
 - **private, max-age=300** — лише на клієнті 5 хв;
 - **no-store** — не кешувати зовсім.
- **ETag**:
 - ідентифікатор версії відповіді;
 - з **If-None-Match** → 304, якщо не змінилось.

Нотатки доповідача:

Поясніть, що для фінансового API частіше доречно `private` й короткий `max-age` (щоб не «залипали» суми).

Слайд 9а - Практичний приклад: Блок 3 (HTTP-кеш для списків і довідників)

На екрані:

- Заголовок: «Практика: Cache-Control для акаунт-платежів і довідників»
- Підзаголовок: «Короткий `private` для персональних, довгий `public` для спільних»
- → **Демонстрація:** перейти до `lesson_9_3_practice.md`, розділ «1. Блок 3: HTTP-кеш → Слайд 9а». Показати:
 - приклад `Cache-Control: private, max-age=15` для списку платежів акаунта;
 - приклад `Cache-Control: public, max-age=3600` для довідника валют/тарифів.

Нотатки доповідача:

Поясніть, як навіть 10–30 секунд `private`-кешу розвантажують повторні запити UI.

БЛОК 4: Контроль об'єму відповіді — DTO/Resources, пагінація

Слайд 10 - Об'єм JSON теж має значення

На екрані:

- Заголовок: «Менше полів → менше байтів → швидше»
- Ідеї:
 - DTO/Resources визначають набір полів;
 - зайві поля = зайвий трафік і серіалізація;
 - особливо критично для великих списків і звітів.

Нотатки доповідача:

Зв'яжіть із Модулем 7: якісні DTO/Resources — це й архітектура, й перформанс.

Слайд 11 - Пагінація як обов'язковий інструмент

На екрані:

- Заголовок: «Списки без пагінації недовго живуть»
- Пункти:
 - обмежує розмір відповіді;
 - стабілізує навантаження;
 - must-have для платежів, транзакцій, звітів.

Нотатки доповідача:

Нагадайте приклади з 9.1–9.2: списки платежів/виписок — перший кандидат на пагінацію.

Слайд 11a - Практичний приклад: Блок 4 (аналіз обсягу однієї відповіді)

На екрані:

- Заголовок: «Практика: рахуємо, що віддає endpoint»
- Підзаголовок: «Скільки полів/байт і що можна прибрати»
- → **Демонстрація:** перейти до `lesson_9_3_practice.md`, розділ «2. Блок 4: Об'єм відповіді →

Слайд 11a». Попросити:

- оцінити кількість полів і розмір JSON для одного з їхніх endpoint'ів;
- запропонувати, які поля прибрати або винести в інший ресурс.

Нотатки доповідача:

Підкресліть, що це дає вигреш і в мережі, і в часі серіалізації.

БЛОК 5: Rate limiting — захист і передбачуваність навантаження

Слайд 12 - Що таке rate limiting

На екрані:

- Заголовок: «Ліміти запитів — захист для всіх»
- Ідеї:
 - обмежує частоту запитів від клієнта;
 - захищає від зловживань і надмірного навантаження з боку скомпрометованих клієнтів;
 - особливо важливий для публічних API й важких операцій.

Нотатки доповідача:

Поясніть, що rate limiting — не «кара для фронту», а захист системи й інших користувачів.

Слайд 13 - Де потрібні жорсткіші ліміти

На екрані:

- Заголовок: «Не всі endpoint'и рівні»
- Приклади:
 - авторизація (захист від брутфорсу);
 - експорт звітів/виписок;
 - heavy-операції з агрегації.

Нотатки доповідача:

Зв'яжіть із `throttle` в Laravel та аналогами в Symfony.

Слайд 13a - Практичний приклад: Блок 5 (налаштування throttle в Laravel)

На екрані:

- Заголовок: «Практика: обмежуємо частоту запитів до API»

- Підзаголовок: «60 req/min для звичайних, 5 req/min для важких»
- → **Демонстрація:** перейти до `lesson_9_3_practice.md`, розділ «3. Блок 5: Rate limiting → Слайд 13а». Показати:
 - приклад `throttle:60,1` для стандартних endpoint'ів;
 - жорсткіший ліміт для експорту/звітів.

Нотатки доповідача:

Поясніть, як фронт може коректно обробляти 429 (retry/backoff, повідомлення користувачу).

БЛОК 6: Jobs, асинхронність і highload

Слайд 14 - Складна логіка в HTTP-запиті = повільне API

На екрані:

- Заголовок: «Чого не варто робити в одному запиті»
- Приклади:
 - генерація PDF-звіту;
 - побудова складної статистики за всіма рахунками;
 - масова обробка транзакцій.

Нотатки доповідача:

Зв'яжіть із Модулем 5: для таких завдань у нас уже є jobs і воркери.

Слайд 15 - Jobs як частина оптимізації API

На екрані:

- Заголовок: «API швидко відповідає, воркер робить важке»
- Ідея:
 - API приймає запит і додає job у чергу;
 - повертає 202 + task_id або посилання;
 - фронт опитує статус або отримує сповіщення.

Нотатки доповідача:

Поясніть, як це радикально зменшує ризик таймаутів і підвищує відгукливість UI.

Слайд 15а - Практичний приклад: Блок 6 (асинхронний експорт виписки)

На екрані:

- Заголовок: «Практика: виносимо експорт виписки в job»
- Підзаголовок: «Синхронно vs асинхронно»
- → **Демонстрація:** перейти до `lesson_9_3_practice.md`, розділ «4. Блок 6: Jobs для важких операцій → Слайд 15а». Попросити:
 - описати поточну реалізацію (якщо є) і запропонувати модель із job;
 - оцінити очікуваний виграш за часом відповіді API.

Нотатки доповідача:

Можна нагадати, що це продовження ідей з Модулів 5 і 9.2 (кеш/агрегації).

БЛОК 7: Highload і горизонтальне масштабування

Слайд 16 - Підготовка до highload (концептуально)

На екрані:

- Заголовок: «Не “вижати максимум з одного серверу”, а вміти масштабуватися»
- Ідеї:
 - кеш → менше навантаження на БД;
 - асинхронність → швидкі відповіді;
 - контроль обсягу й ліміти → передбачуване навантаження;
 - stateless API → можна додавати інстанси за балансувальником.

Нотатки доповідача:

Підкресліть, що highload — це не обов'язково мільйони RPS; це «більше, ніж сьогодні».

Слайд 17 - Stateless API й горизонтальне масштабування

На екрані:

- Заголовок: «Що потрібно для кількох екземплярів застосунку»
- Вимоги:
 - автентифікація за токеном/cookie без прив'язки до конкретного сервера;
 - сесії/кеш/черги в зовнішніх сховищах (Redis, БД);
 - відсутність локального стану, від якого залежать запити.

Нотатки доповідача:

Поясніть, що це фундамент для будь-якого “scale-out”; деталі DevOps виносьте за межі програми.

БЛОК 8: Підсумок, інтерактив і завершення програми

Слайд 18 - Типові помилки під час «підготовки до highload»

На екрані:

- Заголовок: «Як можна марно ускладнити життя собі й команді»
- Приклади:
 - складні рішення без вимірювань;
 - мікросервіси «на майбутнє»;
 - ігнорування HTTP-кешу й пагінації;
 - відсутність rate limiting;
 - «віддавати все й одразу» в одній відповіді.

Нотатки доповідача:

Попросіть учасників згадати, де їм уже зустрічалися такі «зайві ускладнення».

Слайд 19 - Production-ready API: чек-лист

На екрані:

- Заголовок: «Що має бути в «живому» API»
- Пункти:
 - оптимізована БД (індекси, відсутність N+1);
 - кеш (серверний, де потрібно) + HTTP-кеш для ресурсів, що читаються часто;
 - пагінація всіх списків;
 - rate limiting для захисту;
 - jobs для важких операцій;
 - тести, логування, Sentry, лог-агрегація.

Нотатки доповідача:

Порадьте використовувати це як чек-лист під час рев'ю будь-якого нового чи наявного API.

Слайд 20 - Підсумки всієї програми

На екрані:

- Заголовок: «Що ви тепер умієте»
- Коротке нагадування:
 - Модулі 1–4: PHP/ООП/БД/Laravel;
 - Модуль 5: асинхронність, jobs;
 - Модуль 6: архітектура, якість коду, тести;
 - Модуль 7: REST API, DTO/Resources, документація, API-тести;
 - Модуль 8: логи, Sentry, лог-агрегація, інциденти;
 - Модуль 9: продуктивність (кеш, БД, HTTP-кеш, highload).

Нотатки доповідача:

Зробіть невеликий емоційний акцент: це вже дуже гарний набір для рівня middle у реальних проєктах.

Слайд 21 - Інтерактив: план дій для вашого API

На екрані:

- Заголовок: «Що ви зробите першим ділом після програми?»
- Завдання (з практики):
 - для свого проєкту виписати, що вже є (пагінація, кеш, логи, тести, Sentry...);
 - обрати 2–3 пріоритетні покращення (наприклад, кеш довідників, eager loading, rate limit).

Нотатки доповідача:

Дайте 5–7 хвилин, попросіть кількох учасників поділитися планом; це добре закріплює матеріал.

Слайд 22 - Красна нить: головний висновок програми

На екрані:

- Заголовок: «Production-ready бекенд — це не одна технологія»
- Великий текст:
 - **«Надійний фінансовий бекенд = архітектура + чистий код + тести + API-контракт + логи/моніторинг + продуктивність. Кожен елемент важливий; разом вони дають систему, якій можна довіряти.»**

Нотатки доповідача:

Подякуйте групі за роботу, підкресліть, що тепер у них є каркас, на який можна «навішувати» будь-який домен чи фреймворк.